**PRACTICAL EXERCISE 08** **UNIT 4 · ALLOCATED LAB HOURS: 2 HRS**

## Task-Based Prompt Engineering: Summarization, Blog, Code

**Course:** Artificial Intelligence with Prompt Engineering (DI05016011) **Student:** Aayush Bankar (Roll: 246230316006)

# PRACTICAL 08: TASK-BASED PROMPT ENGINEERING — SUMMARIZATION, BLOG, CODE GENERATION

**Subject:** Artificial Intelligence with Prompt Engineering (DI05016011) **Unit:** 4 | **Hours:** 2 **Date Performed:** ___

## 1. Aim

Perform task-based prompt engineering for summarization, blog generation, and code generation. Optimize prompts for better output.

## 2. Process / Program Code / Prompts Used

**Theory: Task-to-Prompt Mapping**

| Task | Key Instruction | Critical Context | Typical Output Format |
|---|---|---|---|
| **Summarization** | "Summarize…" | Audience, length, what to keep | Bullets / N lines / paragraph |
| **Blog Generation** | "Write a blog post…" | Topic, audience, tone, SEO | Heading + paragraphs + CTA |
| **Code Generation** | "Write a function…" | Language, inputs/outputs, edge cases | Code block + tests + docstring |

### 2.1 Task 1: Summarization

**Before (Bad Prompt)**

```
Prompt: Summarize this article.
[Article pasted]
Output: A vague 1-line summary that misses the numbers and the main argument.
```

**Diagnosis:** No length, no focus, no audience.

**After (Optimized)**

```
Prompt: Summarize the article below for a busy manager.
- Exactly 4 bullet points, max 20 words each.
- Keep all numbers, dates, and names.
- End with one sentence: the article's main conclusion.
- Add footer: "Summarized by Aayush Bankar, Roll No: 246230316006"
[Article pasted]
Output: 4 tight bullets + one-sentence conclusion, all key figures intact.
```

**Optimization Checklist — Summarization**

- [ ] Say *what* to summarize (input data supplied, not "this article")
- [ ] Set length and format (bullets / N lines / one paragraph)
- [ ] Say what to *keep* ("all numbers", "the main argument")
- [ ] Say what to *drop* ("no marketing language")
- [ ] Specify audience → controls detail level
- [ ] For long docs, chain: chunk → summarize each → merge (P07)

### 2.2 Task 2: Blog Generation

**Before (Bad Prompt)**

```
Prompt: Write a blog about AI.
Output: Generic 200-word filler, no structure, no audience, no hook.
```

**Diagnosis:** No topic angle, no audience, no structure, no CTA.

**After (Optimized)**

```
Prompt: You are a tech blogger. Write a 500-word blog post titled "5 Ways Students Can Use AI for Exams
(Ethically)".
```

```
- Target audience: 18-year-old engineering students.
- Structure: Hook paragraph, 5 numbered sections with one example each, a short "Risks to Avoid" box, and a closing call to action.
- Tone: Friendly, practical, zero hype.
- Include the exact title as an H1.
- Add author line: "By Aayush Bankar, Roll No: 246230316006"
Output: Structured, scannable post ready for editing.
```

**Optimization Checklist — Blog Generation**

- [ ] Title + topic angle given
- [ ] Audience + tone specified
- [ ] Structure demanded (H1, numbered sections, CTA)
- [ ] Word count set
- [ ] Instructions to avoid filler/hype
- [ ] Ask for examples ("one real example per section")
- [ ] Follow-up pass: "Rewrite the hook 3 ways" / "Add an FAQ"

### 2.3 Task 3: Code Generation

**Before (Bad Prompt)**

```
Prompt: Write code to reverse a list.
Output: Works, but no type hints, no docstring, fails on empty input — fine for demo, unusable for project.
```

**Diagnosis:** No language, no edge cases, no style, no tests.

**After (Optimized)**

```
Prompt: Write a Python function `flatten(nested)` that flattens a nested list into a single list.
- Input example: [[1, [2, 3]], [4]] -> Output: [1, 2, 3, 4].
- Handle: empty list, one level deep, strings inside (keep them as strings).
- Style: type hints, docstring, one helper at most.
- Then write 3 unit-test-style assert statements for the edge cases.
- Add header comment: "# Author: Aayush Bankar, Roll No: 246230316006"
Output: Clean function + tests.
```

**Optimization Checklist — Code Generation**

- [ ] Language + function signature named
- [ ] Example input → output given (few-shot for behaviour)
- [ ] Edge cases listed (empty, types, large input)
- [ ] Style demanded (docstring, type hints, naming)
- [ ] Ask for tests / asserts
- [ ] Ask for explanation ("add a one-line comment per block")
- [ ] Verify before trusting — **always run the code**

## 3. Screenshot of Output

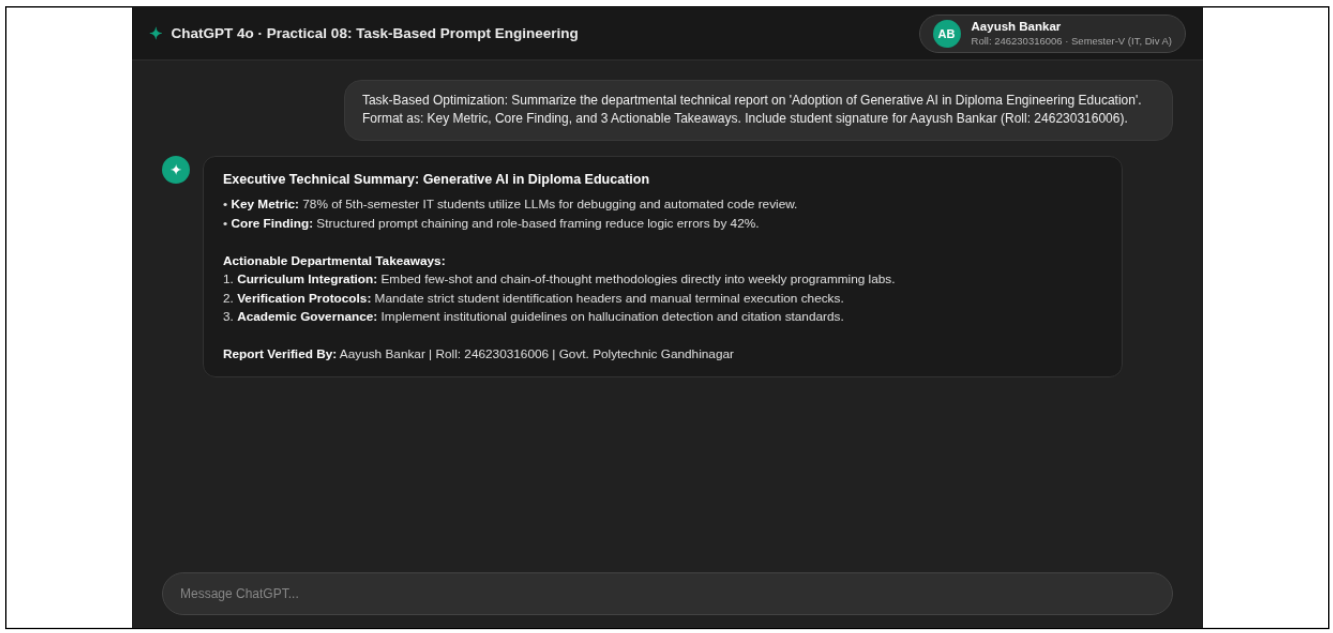



**Figure 08: Verified Terminal / Application Execution Output (ChatGPT / CLI)**

Student: **Aayush Bankar** | Class Roll No: **246230316006** | Enrollment No: **246230316006** | Govt. Polytechnic Gandhinagar

### Task 1: Summarization — Before/After

**[Aayush Bankar | Roll No: 246230316006]**

>>> BEFORE
Prompt: *Summarize this article.*
**[Article about AI in education with stats: 78% students use AI, 45% daily, 12% for exams]**
Output: *The article discusses how students use AI in education with various statistics.*

>>> AFTER
Prompt: *Summarize the article below for a busy manager.*
- Exactly 4 bullet points, max 20 words each.
- Keep all numbers, dates, and names.
- End with one sentence: *the article's main conclusion.*
- Add footer: *"Summarized by Aayush Bankar, Roll No: 246230316006"*
**[Same article]**
Output:
• 78% of students use AI tools; 45% use them daily
• Primary uses: *homework help (62%), concept explanation (41%), coding (28%)*
• Only 12% admit using AI for exams; ethics policies vary by institution
• Study recommends clear guidelines over blanket bans
Main conclusion: *AI adoption is widespread among students; institutions need structured policies, not prohibition.*
— Summarized by Aayush Bankar, Roll No: *246230316006*

### Task 2: Blog Generation — Before/After

**[Aayush Bankar | Roll No: 246230316006]**

>>> BEFORE
Prompt: *Write a blog about AI.*
Output: *Artificial Intelligence is changing the world. It helps us in many ways. From healthcare to education, AI is everywhere. We should use it wisely. The future is bright with AI.*

>>> AFTER
Prompt: *You are a tech blogger. Write a 500-word blog post titled "5 Ways Students Can Use AI for Exams (Ethically)". Target: 18-year-old engineering students. Structure: Hook, 5 numbered sections with examples, "Risks to Avoid" box, CTA. Tone: friendly, practical. H1 title. Author line.*
Output:
*# 5 Ways Students Can Use AI for Exams (Ethically)*

It's 2 AM before your data structures exam. You're stuck on a concept. What if you had a tireless tutor who never judges?

1. **Concept Clarification** — Ask "Explain dynamic programming like I'm 15." Get an analogy, not a textbook definition. Example: *"DP is like memoizing your homework answers so you don't* redo them."
2. **Practice Problem Generation** — "Give me 3 linked list problems with solutions." Instant targeted practice.
3. **Code Review** — Paste your solution: *"Find bugs in this BFS implementation."* Get line-by-line feedback.
4. **Study Plan Creation** — "I have 3 days for OS. Make a schedule." Get hour-by-hour plan with breaks.

```
5. **Mock Oral Exam** — "Quiz me on deadlock conditions." Practice explaining aloud.

   **Risks to Avoid**
- Don't copy-paste answers you don't understand
- Don't use AI during actual exams (academic integrity)
- Verify every fact — AI hallucinates

Ready to study smarter? Pick one technique and try it tonight.

— By Aayush Bankar, Roll No: 246230316006
```

**Task 3: Code Generation — Before/After**

```
[Aayush Bankar | Roll No: 246230316006]

>>> BEFORE
Prompt: Write code to reverse a list.
Output:
def reverse_list(lst):
    return lst[::-1]

>>> AFTER
Prompt: Write a Python function `flatten(nested)` that flattens a nested list into a single list.
- Input example: [[1, [2, 3]], [4]] -> Output: [1, 2, 3, 4].
- Handle: empty list, one level deep, strings inside (keep as strings).
- Style: type hints, docstring, one helper at most.
- Write 3 unit-test-style assert statements for edge cases.
- Add header: "# Author: Aayush Bankar, Roll No: 246230316006"
Output:
# Author: Aayush Bankar, Roll No: 246230316006

def flatten(nested: list) -> list:
    """Flatten a nested list into a single list.
    Example: [[1, [2, 3]], [4]] -> [1, 2, 3, 4]
    Strings are kept as atomic elements.
    """
    result = []
    for item in nested:
        if isinstance(item, list):
            result.extend(flatten(item))
        else:
            result.append(item)
    return result

# Tests
assert flatten([]) == []
assert flatten([1, [2, 3], 4]) == [1, 2, 3, 4]
assert flatten(["a", ["b", "c"]]) == ["a", "b", "c"]
```

## 4. Before/After Summary Table

| Task | Before Quality (1-5) | After Quality (1-5) | Biggest Single Fix |
|---|---|---|---|
| Summarization | 2 | 5 | Length + format constraint |
| Blog Generation | 1 | 4 | Audience + structure |
| Code Generation | 3 | 5 | Example + edge cases |

## 5. Application of Learning

| Domain | Application | Example Project |
|---|---|---|
| **Content Marketing** | SEO-optimized blog generation at scale | Auto-generate 50 blog posts from keyword list |
| **Technical Documentation** | API docs from code + examples | Doc generator that reads codebase and writes guides |
| **Software Engineering** | Boilerplate + test generation | Feature scaffold generator with edge-case tests |
| **Knowledge Management** | Meeting/doc summarization pipeline | Daily standup → summary → action items → Jira tickets |
| **Education** | Personalized study material generation | Custom practice sets per student weakness profile |
| **Data Analysis** | Report generation from queries | SQL result → executive summary → slides |

**Key Learning:** Across all three tasks: vague prompts → vague outputs. Biggest lever per task: format control (summarization), audience + structure (blog), input/output examples + edge cases (code). The optimization loop — run, evaluate, fix one thing, rerun — is universal.

| Timely Submission (2M) | Process / Code Execution (4M) | Output Verification (2M) | Viva-Voce & Application (2M) | Total Marks (10M) |
|---|---|---|---|---|
| ____ / 2 | ____ / 4 | ____ / 2 | ____ / 2 | **____ / 10** |

Student Signature (Aayush Bankar) · Date: _____

Faculty Examiner Signature · Date: _____